Section 4.1: Why Convolutional Networks?

The Limits of the MLP and Three Key Ideas

AI-IM Class — Lecture Notes

In Section 3.4 we built a complete classifier: a stack of fully connected layers, a softmax output and the cross-entropy loss. Nothing in that recipe says what the input features *are*; an MLP accepts any vector $\mathbf{x}$. In this chapter the input is an **image**. We will see that an MLP *can* read an image, but that it does so in a very wasteful way, and that three simple ideas fix the problem. Those three ideas define the **convolutional neural network (CNN)**. In this section we

- describe how an image is stored as numbers,
- feed an image to an MLP and find three weaknesses,
- introduce the three ideas behind convolution: *sparse interactions*, *parameter sharing* and *equivariance*, and
- look briefly at where the idea came from.

1 Images as Data

A **grayscale image** is a grid of numbers. Each number is a **pixel** and gives the brightness at that location, usually an integer from 0 (black) to 255 (white). Before training we normally rescale the pixels to $[0, 1]$ by dividing by 255.

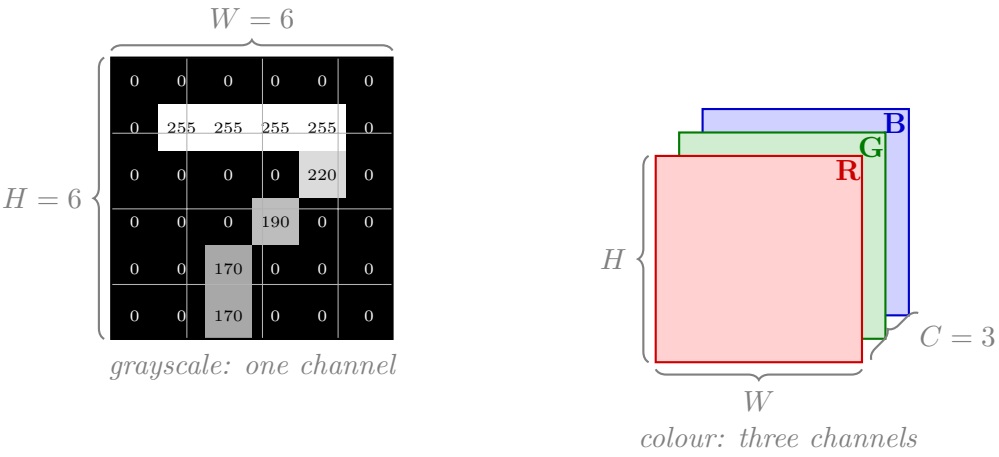


Figure 1: Left: a 6×6 grayscale image of the digit "7" and its pixel values. Right: a colour image stores three such grids, one per *channel* (red, green, blue).

A **colour image** stores three grids of the same size: one each for the red, green and blue intensity. Each grid is a **channel**. So an image is a three-dimensional array (a *tensor*) with

- C channels ($C = 1$ for grayscale, $C = 3$ for colour),
- height H (number of rows), and
- width W (number of columns).

We write the image as $\mathbf{X}$ and its entries in lower case: $x_{i,j}$ is the pixel in row i and column j (row 1 is the top row).

PyTorch convention. PyTorch stores one image as a tensor of shape (C, H, W), channels first. A mini-batch of N images (Section 3.4) has shape (N, C, H, W). A batch of 64 colour images of size 224×224 is therefore a tensor of shape $(64, 3, 224, 224)$. We use this order throughout the chapter.

Exercise 1 How big is an image?

How many numbers are stored in each of the following?

(a) One MNIST digit, a grayscale image of 28×28 pixels: $28 \times 28 = 784$

(b) One colour image of 224×224 pixels (a common input size for ImageNet models): $3 \times 224 \times 224 = 150\,528$

(c) A batch of shape $(64, 3, 224, 224)$: $64 \times 150\,528 = 9\,633\,792$

2 Feeding an Image to an MLP

An MLP expects a vector, so the obvious approach is to **flatten** the image: read the pixels row by row and put them in one long column. A 28×28 digit becomes a vector $\mathbf{x}$ with 784 entries, and the MLP from Section 3.4 can classify it with 10 softmax outputs. This works reasonably well on MNIST (about 98% test accuracy). It has three weaknesses, and all three get worse as images become larger and more varied.

2.1 Weakness 1: Too Many Parameters

Recall from Section 3.4 that a fully connected layer with $n_{\ell-1}$ inputs and n_ℓ nodes has $(n_{\ell-1}+1)\,n_\ell$ parameters. With an image as input, n_0 is the number of pixel values, which is large.

Exercise 2 Parameters of the first layer

Count the parameters of the *first* hidden layer only.

Input	Inputs n_0	Hidden nodes n_1	Parameters
MNIST, $1 \times 28 \times 28$	784	128	100 480
Colour, $3 \times 224 \times 224$	150 528	1 000	150 529 000

The second row has about **150 million** parameters in one layer. For comparison, ResNet-50, a famous CNN that we meet in Section 4.4, has about 25 million parameters in its *entire* network. Every parameter costs memory and computation, and, more importantly, must be learned from data. More parameters means more training images are needed before the model stops fitting noise.

2.2 Weakness 2: Flattening Ignores Which Pixels Are Neighbours

In an image, the pixels next to each other (left, right, above, below) usually belong to the same object: an edge, a stroke, a corner. Flattening destroys this layout. Two pixels that are vertically adjacent end up far apart in the vector.

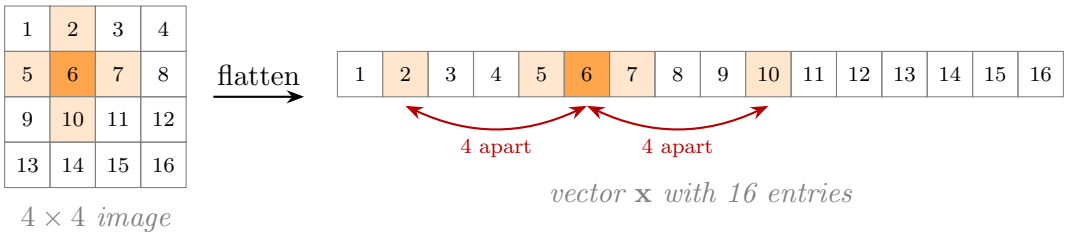


Figure 2: Pixel 6 (dark) and its four neighbours (light). After flattening, only the left and right neighbours stay next to it. The neighbours above and below are a whole row (W entries) away.

Exercise 3 Where does a pixel go?

A 28×28 image is flattened row by row (this is what `torch.flatten` does). Number the entries of $\mathbf{x}$ from 1.

(a) The pixel in row i, column j goes to position $(i-1) \cdot 28 + j$.

(b) Pixel $(3, 5)$ goes to position 61. Its right neighbour $(3, 6)$ goes to 62, and the pixel below it, $(4, 5)$, goes to 89.

The MLP does not use the layout at all. A thought experiment makes this clear: pick one fixed random reordering of the 784 pixels and apply it to every training and test image. To a human the images now look like noise. The MLP, however, trains to *exactly* the same accuracy, because a fully connected layer treats its inputs as an unordered list. Reordering the inputs only reorders the columns of $\mathbf{W}^{(1)}$. Everything we know about images (that nearby pixels go together) is thrown away and must be learned again from data.

2.3 Weakness 3: A Pattern Learned in One Place Is Unknown Elsewhere

A cat is a cat whether it sits on the left or the right of the photo. For an MLP, the two cases are unrelated. We show this with the smallest possible example: a one-dimensional "image" with seven pixels, containing one bright pixel (a "bump").

$$\mathbf{x}_A = (0,\ 0,\ 1,\ 0,\ 0,\ 0,\ 0), \qquad \mathbf{x}_B = (0,\ 0,\ 0,\ 0,\ 1,\ 0,\ 0).$$

Image B is image A shifted two pixels to the right. Suppose a hidden node has learned to detect the bump in image A, with weights

$$\mathbf{w} = (0,\ -1,\ 2,\ -1,\ 0,\ 0,\ 0), \qquad b = 0.$$

The weight $+2$ rewards a bright centre pixel; the two -1 weights penalise bright neighbours, so the node responds to an isolated bright point.

Exercise 4 One node, two positions

Compute the pre-activation $z = \mathbf{w}^\top \mathbf{x} + b$ for both images.

$$z_A = (-1)(0) + (2)(1) + (-1)(0) = 2, \qquad z_B = 0$$

The node fires for image A and is silent for image B, although both contain the same bump. To detect the bump at every position, the MLP needs a separate node for each position, each learning the *same* pattern $(-1, 2, -1)$ from its own examples. A CNN instead learns the pattern once and reuses it everywhere.

3 Three Key Ideas

The fix for the bump problem is to take the small pattern $\mathbf{w} = (w_1, w_2, w_3) = (-1, 2, -1)$, called a **kernel** (or **filter**), and slide it along the input. At each position we compute one weighted sum:

$$z_i = w_1 x_i + w_2 x_{i+1} + w_3 x_{i+2}, \qquad i = 1, 2, \ldots$$

This sliding weighted sum is the **convolution** that gives CNNs their name. Section 4.2 defines it formally, including what happens at the borders. For now we only need the formula above. With 7 inputs and a kernel of width 3 there are $7 - 3 + 1 = 5$ positions, so we get 5 outputs $z_1, \ldots, z_5$.

Compared with a fully connected layer, this layer has three properties that make it a good fit for images (Goodfellow et al., *Deep Learning*, Section 9.2).

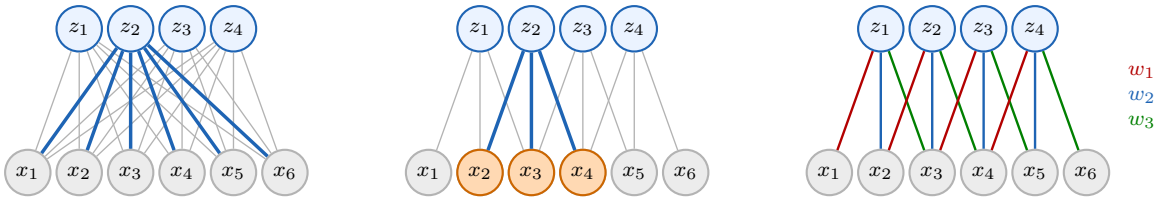
3.1 Idea 1: Sparse Interactions

In a fully connected layer, every output is connected to every input. In a convolution, output z_i is connected to only k inputs, where k is the width of the kernel. The inputs that affect an output form its **receptive field**. In Figure 3 the receptive field of z_2 is $\{x_2, x_3, x_4\}$.

This matches what we know about images: small, useful features such as edges and corners involve only a few neighbouring pixels, not the whole image.

3.2 Idea 2: Parameter Sharing

In a fully connected layer every weight is used exactly once: it multiplies one input to produce one output. In a convolution the *same* k weights are used at every position. This is **parameter sharing** (also called *tied weights*). The layer learns one kernel instead of one set of weights per location.



(a) dense (b) local (c) local + shared

Figure 3: Six inputs, four outputs. (a) A dense layer connects every output to every input. (b) Local connections: z_2 sees only its receptive field x_2, x_3, x_4. (c) Convolution: local connections that reuse the same three weights at every position (same colour = same weight).

Exercise 5 Counting weights

Count the weights (ignore biases) for the three layers in Figure 3, then for general sizes: m inputs, n outputs and kernel width k.

Layer	$m = 6,\ n = 4,\ k = 3$	General
(a) Dense	24	mn
(b) Local	12	kn
(c) Local + shared	3	k

Sparse interactions reduce the weights from mn to kn; sharing reduces them further to k, which no longer depends on the size of the image. The following example from Goodfellow et al. shows how large the difference is for a real photo.

Exercise 6 Edge detection

The kernel $\mathbf{w} = (-1, 1)$ computes $z_i = x_{i+1} - x_i$: each pixel minus its left neighbour.

(a) Apply it to one row of an image, $\mathbf{x} = (10, 10, 10, 80, 80, 80)$:

$$\mathbf{z} = \big(0,\ 0,\ 70,\ 0,\ 0\big)$$

The output is non-zero only at the jump from dark to bright: the kernel is a **vertical edge detector**.

(b) Now apply it to every row of a photo that is 280 pixels tall and 320 pixels wide. Each row gives 319 outputs, so the output image has $280 \times 319 = 89\,320$ pixels.

(c) Each output costs two multiplications and one addition. The whole convolution costs $267\,960$ operations and has only 2 parameters.

(d) A dense layer that computes the same output needs one weight per (input, output) pair:

$$(280 \times 320) \times (280 \times 319) = 89\,600 \times 89\,320 \approx 8.0 \times 10^9 \text{ weights.}$$

Over eight billion weights, almost all of them zero, to describe what two numbers describe. Convolution is an extremely compact way to apply *the same small, local operation*

everywhere in an image.

3.3 Idea 3: Equivariance to Translation

Return to the bump images $\mathbf{x}_A$ and $\mathbf{x}_B$ of Section 2.3. Instead of one dense node, use the sliding kernel $\mathbf{w} = (-1, 2, -1)$.

Exercise 7 Slide the kernel

Compute $z_i = -x_i + 2x_{i+1} - x_{i+2}$ for $i = 1, \ldots, 5$.

i	1	2	3	4	5	6	7
$x_{A,i}$	0	0	1	0	0	0	0
$z_{A,i}$	-1	2	-1	0	0	—	—
$x_{B,i}$	0	0	0	0	1	0	0
$z_{B,i}$	0	0	-1	2	-1	—	—

The bump in B is two pixels to the right of the bump in A. The output $\mathbf{z}_B$ is $\mathbf{z}_A$ shifted two pixels to the right.

The same kernel finds the bump at both positions, and the output tells us *where* it is. This property has a name.

A function f is **equivariant** to a transformation g if applying g before or after f gives the same result:

$$f\big(g(\mathbf{x})\big) = g\big(f(\mathbf{x})\big).$$

Convolution is equivariant to **translation** (shifting): shift the input, and the output shifts by the same amount.

In two dimensions, the output of a convolution is a **feature map**: a 2-D map showing where a feature (such as a vertical edge) appears in the image. If the object moves, its features move by the same amount in the map.

Equivariance vs. invariance. For classification we eventually want a stronger property. The answer "cat" should not change at all when the cat moves. A function is **invariant** to g if $f(g(\mathbf{x})) = f(\mathbf{x})$. Convolution alone is equivariant, not invariant, but one extra step makes it invariant.

Exercise 8 From equivariant to invariant

Summarise each feature map from Exercise 7 by its largest value:

$$\max_i z_{A,i} = 2, \qquad \max_i z_{B,i} = 2.$$

The summary answers "is there a bump?" and gives the same answer for both positions.

Taking a maximum over a region is called **pooling**, and it is the topic of Section 4.3. The two steps divide the work: convolution detects features wherever they are, and pooling discards their exact position.

What convolution does *not* give. Convolution is equivariant to shifts only. It is not naturally equivariant to rotation or to a change of scale: a kernel that detects vertical edges does not detect horizontal ones. CNNs handle these variations by learning many kernels (Section 4.2) and by training on rotated and rescaled copies of the images.

3.4 Deeper Layers See More

Sparse interactions do not mean that the network only ever looks at small regions. If we stack convolutional layers, each output of the second layer depends on several outputs of the first, which together cover a wider part of the input.

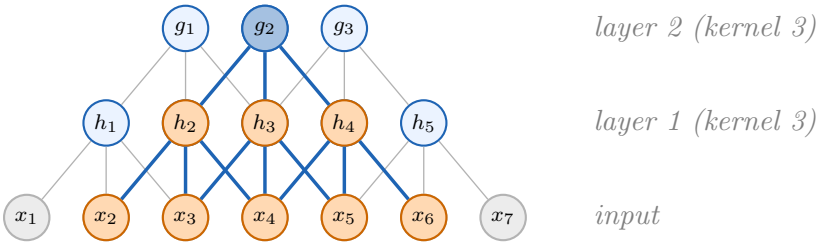


Figure 4: Two stacked convolutions, each with kernel width 3. Unit g_2 is directly connected to only three units of layer 1, but indirectly depends on five inputs, $x_2, \ldots, x_6$.

Exercise 9 Growing receptive field

Each layer uses a kernel of width 3. Fill in the width of the receptive field (measured on the input) of one unit in layer L.

Layer L	1	2	3	10	L
Receptive field	3	5	7	21	$2L+1$

With kernels of width k, each layer adds $k-1$, so the receptive field is $1 + L(k-1)$. How many layers of width 3 are needed before one unit sees a whole MNIST row of 28 pixels? $2L + 1 \geq 28 \Rightarrow L = 14$

Fourteen layers just to see one row is slow growth. Pooling and strides (Sections 4.2 and 4.3) make the receptive field grow much faster. The picture of a deep CNN that emerges is a hierarchy: early layers detect edges in small patches, middle layers combine edges into textures and parts, and late layers, which see most of the image, combine parts into objects.

4 Where the Idea Came From

CNNs were inspired by the **visual cortex**, the part of the brain that processes what the eyes see.

- In the late 1950s and 1960s, David Hubel and Torsten Wiesel recorded single neurons in the visual cortex of cats. Each neuron responded only to stimuli in a small region of the visual field (a *local receptive field*), and many responded only to edges of a particular orientation. *Simple cells* reacted to an edge at an exact position; *complex cells* reacted to the same edge anywhere in a small region. These correspond to convolution and pooling. The work earned the 1981 Nobel Prize in Medicine.
- In 1980 Kunihiko Fukushima built the **Neocognitron**, a network of alternating "simple" and "complex" layers, but without gradient-based training.
- In 1989 Yann LeCun and colleagues trained a convolutional network with backpropagation to read handwritten ZIP codes. This led to **LeNet-5** (1998), which read cheques in US banks.
- In 2012 **AlexNet** won the ImageNet challenge by a wide margin using GPUs, and CNNs became the standard approach to computer vision.

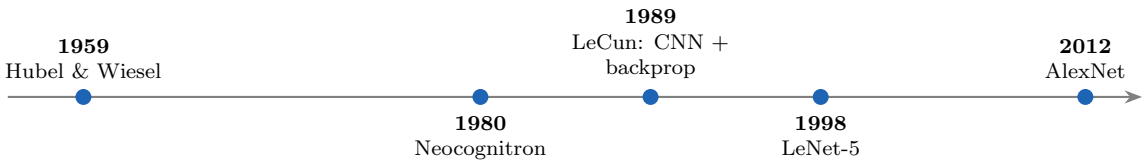


Figure 5: From neuroscience to deep learning.

Inspiration, not imitation. A CNN is not a model of the brain. The useful lesson from biology is the *design*: local receptive fields, the same detector repeated across the visual field, and a hierarchy from simple to complex features. These are exactly Ideas 1–3.

5 Summary

MLP weakness	Idea	Effect
Every pixel connects to every node	Sparse interactions	Each output sees a small receptive field
Separate weights for every position	Parameter sharing	One kernel of k weights, independent of image size
Pattern must be relearned at each position	Equivariance	Shift the input, the feature map shifts with it

- An image is a tensor of shape (C, H, W); a batch has shape (N, C, H, W) in PyTorch.
- Flattening an image into an MLP creates a huge first layer, ignores which pixels are neighbours, and forces every pattern to be learned again at every position.
- A convolution slides one small kernel across the input: $z_i = w_1 x_i + w_2 x_{i+1} + \cdots + w_k x_{i+k-1}$.

- Convolution is equivariant to translation; adding pooling makes the result (approximately) invariant.
- Stacking layers enlarges the receptive field, giving a hierarchy of edges $\to$ parts $\to$ objects.

Coming up. Section 4.2 defines convolution in 1-D and 2-D, with padding, stride and multiple channels, and computes its gradient. Section 4.3 adds pooling and assembles a complete CNN. Section 4.4 studies famous architectures from LeNet-5 to ResNet, and Section 4.5 looks beyond classification at object detection and OCR.